

Resumen

La validación de datos es un aspecto fundamental en el desarrollo de APIs RESTful para garantizar la integridad y consistencia de la información. Implementar validaciones adecuadas en los endpoints de POST y PUT evita errores que podrían comprometer toda la base de datos y asegura que la aplicación funcione correctamente. **Estas prácticas no solo mejoran la calidad del código, sino que también proporcionan una mejor experiencia al usuario final.**

¿Cómo implementar validaciones efectivas en APIs RESTful?

Cuando desarrollamos APIs, es crucial validar los datos que recibimos antes de procesarlos. En este caso, hemos utilizado la inteligencia artificial como herramienta para generar validaciones robustas para nuestros endpoints. **La implementación de estas validaciones nos permite controlar la calidad de los datos que ingresan a nuestro sistema.**

Las validaciones que hemos implementado incluyen:

- Verificación de correos electrónicos mediante expresiones regulares
- Comprobación de longitud mínima para nombres de usuario
- Validación de que los IDs sean numéricos y únicos

Estas validaciones se han encapsulado en funciones reutilizables que pueden ser compartidas entre el backend y el frontend si fuera necesario.

¿Qué funciones de validación debemos implementar?

Las funciones de validación que hemos creado son:

```
// Validación de correo electrónico mediante Regex
function isValidEmail(email) {
  const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
  return emailRegex.test(email);
}

// Validación de nombre (mínimo 3 caracteres)
function isValidName(name) {
  return typeof name === 'string' && name.length >= 3;
}

// Validación de ID (numérico y único)
function isValidId(id, users) {
  const isNumeric = !isNaN(id);
  const isUnique = !users.some(user => user.id === id);
  return isNumeric && isUnique;
}

// Función principal de validación
function validateUser(user, users) {
  const errors = [];
```

```

if (!isValidName(user.name)) {
  errors.push("El nombre debe tener al menos tres caracteres");
}

if (!isValidEmail(user.email)) {
  errors.push("El correo electrónico no es válido");
}

if (!isValidId(user.id, users)) {
  errors.push("El ID debe de ser numérico y único");
}

return {
  isValid: errors.length === 0,
  errors: errors
};
}

```

Estas funciones se guardan en un archivo llamado `validation.js` dentro de la carpeta de utilidades, lo que permite importarlas fácilmente en cualquier parte de nuestra aplicación.

¿Cómo integrar las validaciones en los endpoints?

Para integrar estas validaciones en nuestros endpoints de POST y PUT, necesitamos importar la función principal `validateUser` y utilizarla antes de procesar los datos:

```

import { validateUser } from './utils/validation.js';

// En el endpoint POST
app.post('/users', (req, res) => {
  const newUser = req.body;
  const validation = validateUser(newUser, users);

  if (!validation.isValid) {
    return res.status(400).json({ error: validation.errors });
  }

  // Continuar con la lógica para crear el usuario
  users.push(newUser);
  res.status(201).json(newUser);
});

// En el endpoint PUT
app.put('/users/:id', (req, res) => {
  const updatedUser = req.body;
  const validation = validateUser(updatedUser, users);

  if (!validation.isValid) {
    return res.status(400).json({ error: validation.errors });
  }

  // Continuar con la lógica para actualizar el usuario

```

```
// ...  
});
```

¿Cómo probar las validaciones implementadas?

Para verificar que nuestras validaciones funcionan correctamente, podemos utilizar herramientas como Postman para enviar diferentes tipos de solicitudes a nuestros endpoints:

1. **Caso exitoso:** Enviar datos válidos que cumplan con todas las reglas de validación.
2. **ID duplicado:** Intentar crear un usuario con un ID que ya existe.
3. **Nombre corto:** Enviar un nombre con menos de tres caracteres.
4. **Correo inválido:** Proporcionar un correo electrónico con formato incorrecto.

Al probar estos casos, deberíamos recibir respuestas apropiadas:

- Para casos exitosos: código 201 (Created) y los datos del usuario.
- Para casos con errores de validación: código 400 (Bad Request) y mensajes específicos sobre los errores encontrados.

¿Qué desafíos pueden surgir con estas validaciones?

Un desafío común es la validación en el endpoint PUT. Como se observó en las pruebas, puede haber un conflicto cuando intentamos actualizar un usuario existente, ya que la validación de ID único podría rechazar la operación incorrectamente.

Este es un reto interesante: modificar la función de validación para que permita actualizar un usuario existente sin que la validación de ID único interfiera. Una posible solución sería adaptar la función `isValidId` para que considere el contexto de la operación (creación vs. actualización).

La implementación de validaciones robustas es un paso fundamental para construir APIs confiables y seguras. Estas prácticas no solo previenen errores en los datos, sino que también mejoran la experiencia del usuario al proporcionar mensajes de error claros y específicos. **¿Has enfrentado desafíos similares en tus proyectos? Comparte tus experiencias y soluciones en los comentarios.**

Saludos compañeros;

Para darle solución al bug presentado por el profesor, lo que hice fue dividir la funciones de validación de ID en el archivo de validaciones. Adjunto archivo "validations.js":

```
// Validación de correo electrónico mediante Regex
function isValidEmail(email) {
  // Expresión regular para validar el email.
  const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
  return emailRegex.test(email);
}

// Validación de nombre (mínimo 3 caracteres)
function isValidName(name) {
  return typeof name === 'string' && name.length >= 3;
}

// Validación de la no repetición del número de ID.
function isUniqueId(id, users) {
  const isUnique = users.some(user => user.id === id);
  return !isUnique;
}

// Validación de ID (numérico y único)
function isValidId(id) {
  // Expresión regular para validar que el id es un entero positivo.
  const idRegex = /^[0-9]+$/;
  return typeof id === 'number' && idRegex.test(id);
}

// Función principal de validación
function validateUser(user, users) {
  const errors = [];

  if (!isValidName(user.name)) {
    errors.push("El nombre debe tener al menos tres caracteres");
  }

  if (!isValidEmail(user.email)) {
    errors.push("El correo electrónico no es válido");
  }

  if (!isValidId(user.id)) {
    errors.push("El ID debe de ser un número positivo");
  }

  return {
    isValid: errors.length === 0,
    errors: errors
  };
}

module.exports = {
```

```

    isValidEmail,
    isValidName,
    isUniqueId,
    isValidId,
    validateUser
  });
  ``Con esto dicha validación que se encarga de ver que no se repita la ID solo
  la incluyo en POST, pero no en el PUT.

  ``js
  // Ruta para creacion de usuarios (API).
  app.post('/users', (req, res) => {
    // Capturamos los datos implicitos en el cuerpo de la solicitud.
    const newUser = req.body;

    // Desarrollamos lectura de los datos existentes en el archivo JSON.
    fs.readFile(usersFilePath, 'utf-8', (error, data) => {
      // Manejo de errores
      if (error) {
        return res.status(500).json({
          error: "Error con conexión de datos"
        });
      }
      // Parseamos los datos JSON a Objeto.
      const users = JSON.parse(data);

      // Integramos las funciones de validacion.
      const validationId = isUniqueId(newUser.id, users)
      const validation = validateUser(newUser, users);

      // Validamos los datos recibidos.
      // --> Validamos que nombre exista y tenga al menos 3 caracteres.
      if (!validation.isValid) {
        return res.status(400).json({
          error: validation.errors
        });
      } else if (validationId) {
        return res.status(400).json({
          error: "Esa ID ya esta registrada, ingrese una diferente"
        });
      } else {
        // Agregamos el nuevo usuario al formato JSON existente en el
        // archivo.
        users.push(newUser);
        // Guardamos la información del JSON actualizada en el archivo JSON.
        fs.writeFile(usersFilePath, JSON.stringify(users, null, 2), (error)
=> {
          // Manejo de errores
          if (error) {
            return res.status(500).json({
              error: "Error al guardar el usuario nuevo"
            });
          } else {
            // Respuesta de creacion y agregado del nuevo usuario
            (Codigo de estado HTTP = 201 = CREATED).

```

```

        return res.status(201).json(newUser);
    }
    });
    });
});
// ... js
app.put('/users/:id', (req, res) => {
    // Extraemos el id de la ruta dinamica y lo convertimos de string a int.
    const userId = parseInt(req.params.id, 10);
    // Obtenemos los datos de la solicitud.
    const updateUser = req.body;

    fs.readFile(usersFilePath, 'utf8', (error, data) => {
        if (error) {
            return res.status(500).json({
                error: "Error con conexión de datos"
            });
        }
        // Parseamos los datos JSON a Objeto.
        let users = JSON.parse(data);

        // Integramos las funciones de validacion.
        const validation = validateUser(updateUser, users);

        // Validamos los datos recibidos.
        if (!validation.isValid) {
            return res.status(400).json({
                error: validation.errors
            });
        } else if (updateUser.id !== userId) { // Validamos que la ID de la
solicitud y la ID del EndPoint sean iguales
            return res.status(400).json({
                error: "La ID en el EndPoint de la solicitud y la ID del formato
JSON deben ser iguales"
            });
        } else {
            // Recorremos todos los usuarios y actualizar solo el que coincida
con el ID proporcionado.
            users = users.map(user => {
                // Actualizamos el usurio que coincide con el id.
                return user.id === userId ? { ...user, ...updateUser } : user;
            });
            // Guardamos los cambios en el archivo JSON.
            fs.writeFile(usersFilePath, JSON.stringify(users, null, 2), (error)
=> {
                // Manejamos errores
                if (error) {
                    return res.status(500).json({
                        error: "Error al actualizar el usuario"
                    });
                } else {
                    res.status(200).json(updateUser);
                }
            });
        }
    });
});
}

```

```
});  
});  
`Además que agregue un condicional para que las ID las solicitudes PUT tengan  
que coincidir con la ID en el cuerpo del EndPoint.
```